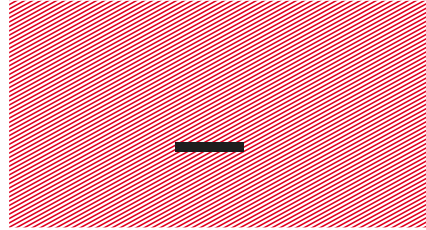


— IoT Devices

Les 4 – I²C





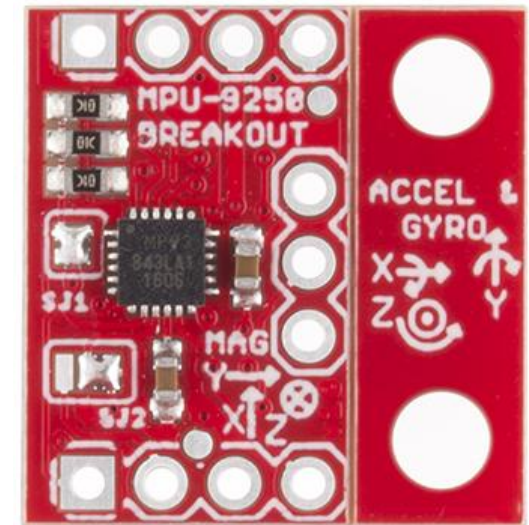
MPU9250

algemene info



MPU-9250

- De MPU-9250 is een **Inertia Measurement Unit** (IMU). Daarmee kan je hoeksnelheid ($^{\circ}/s$) en lineaire versnelling (m/s^2) meten.
- Dergelijk soort sensoren wordt veel gebruikt in **GSM's** om te detecteren hoe het scherm georiënteerd is. Maar ook in **drones** om zo stabiel te kunnen vliegen.
- Je kan deze sensor uitlezen via de **I²C-bus**.
- De sensor is beschikbaar op 'breakout board' (Sparkfun).
- **Wat weet je nog** van I²C van vorig jaar?

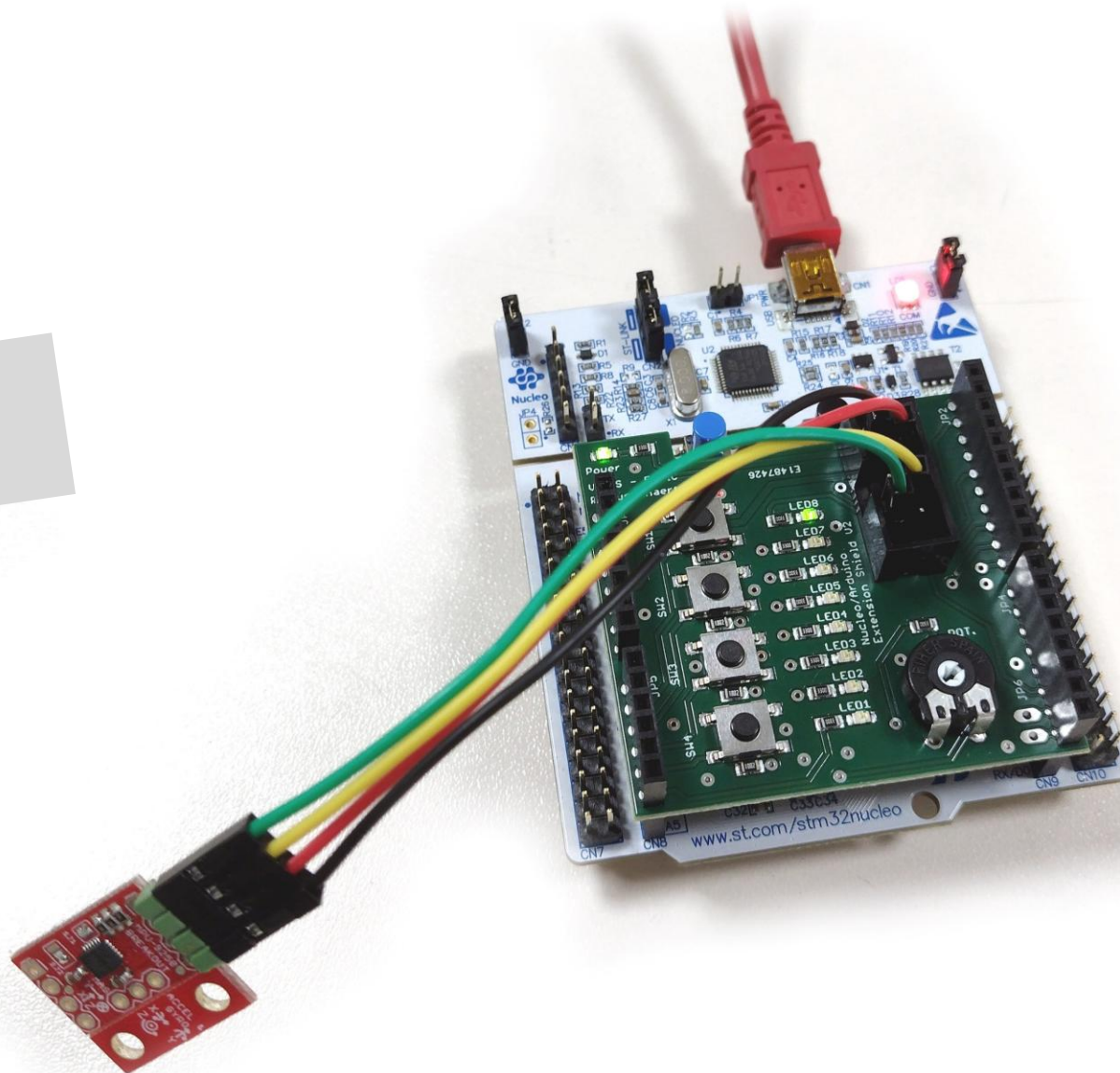


<https://www.sparkfun.com/products/retired/13762>

—

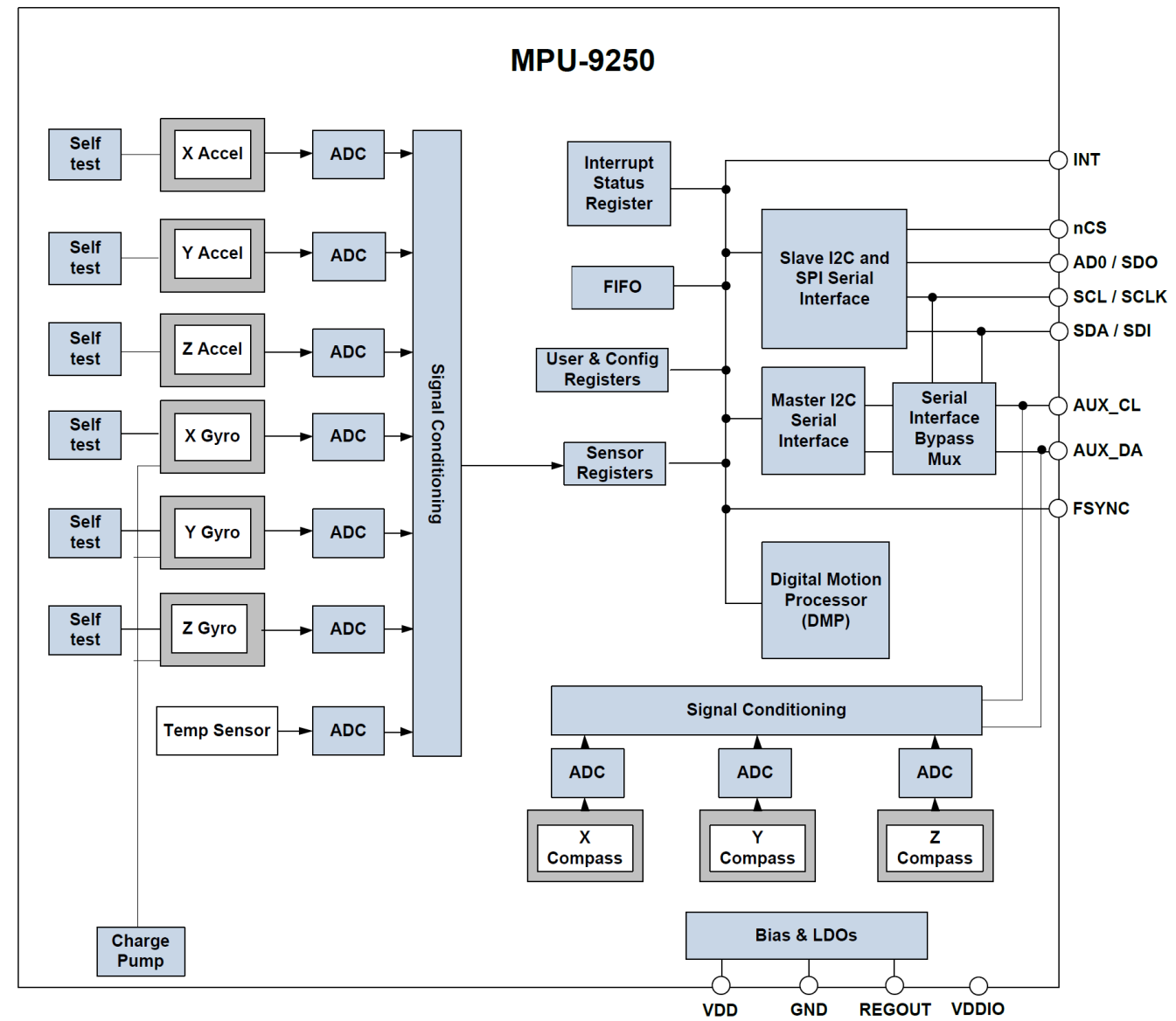
MPU-9250

Je kan de sensor verbinden met 'jumper wires'. Let erop dat je de **GND en 3V3 correct aansluit!**



Zoek de accelerometer en gyroscoop voor de drie assen...

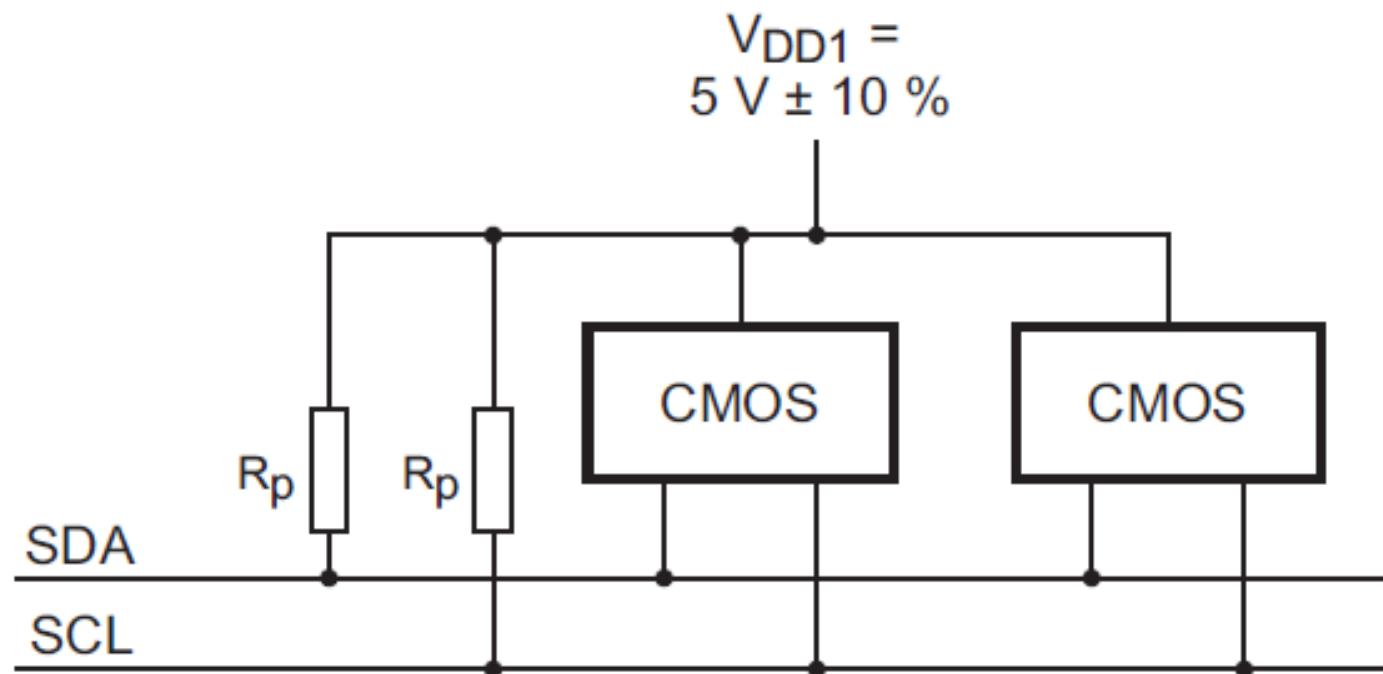
MPU-9250

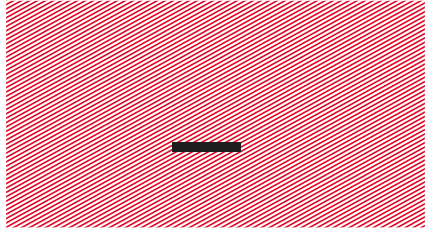


MPU-9250

Let op de **pull-up weerstanden**! Ze zorgen ervoor dat de SDA en SCL, door verschillende gebruikers veilig 'laag' gemaakt kunnen worden zonder dat er schade ontstaat door te hoge stromen. Alle gebruikers moeten wel werken met een 'open collector' of '**open drain**' output.

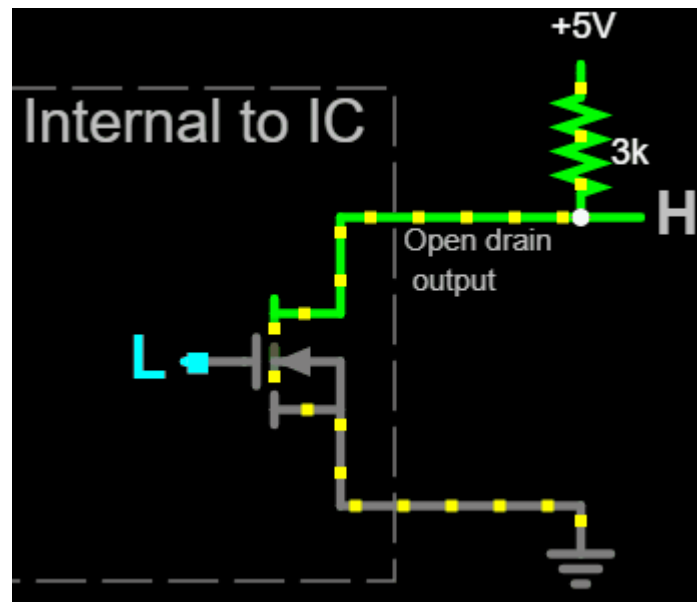
OPM: hier wordt 5V als busspanning aangegeven, wij gaan echter **meestal 3,3V gebruiken**!





MPU-9250

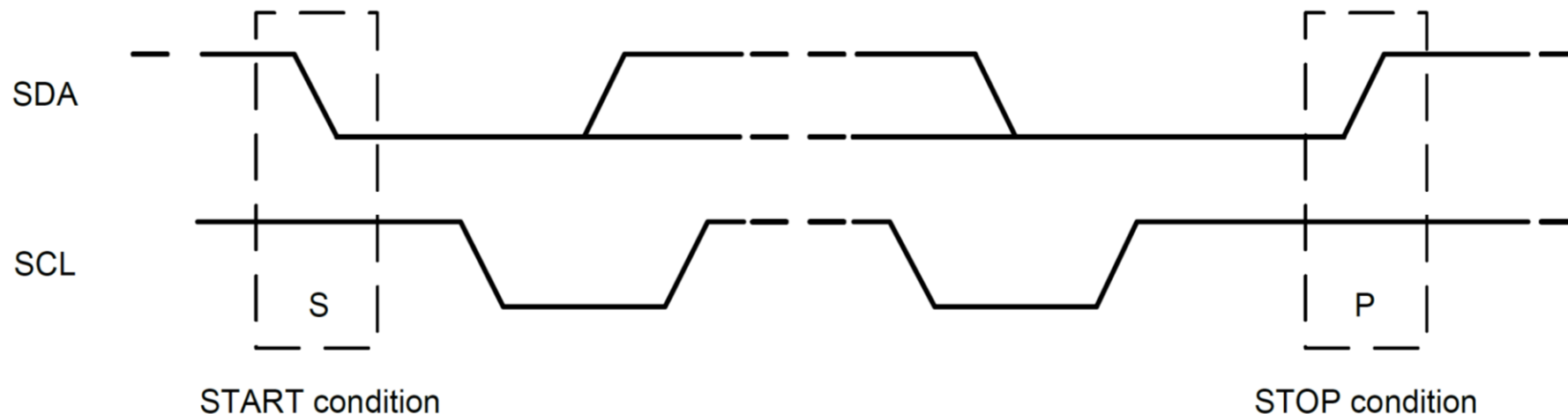
De **werking van een open drain output**, wordt heel bevattelijk voorgesteld op Wikipedia. Heel dikwijls wordt voor 4k7 Ohm gekozen als pull-up.



https://upload.wikimedia.org/wikipedia/commons/4/47/Animated_open_drain_output.gif

MPU-9250

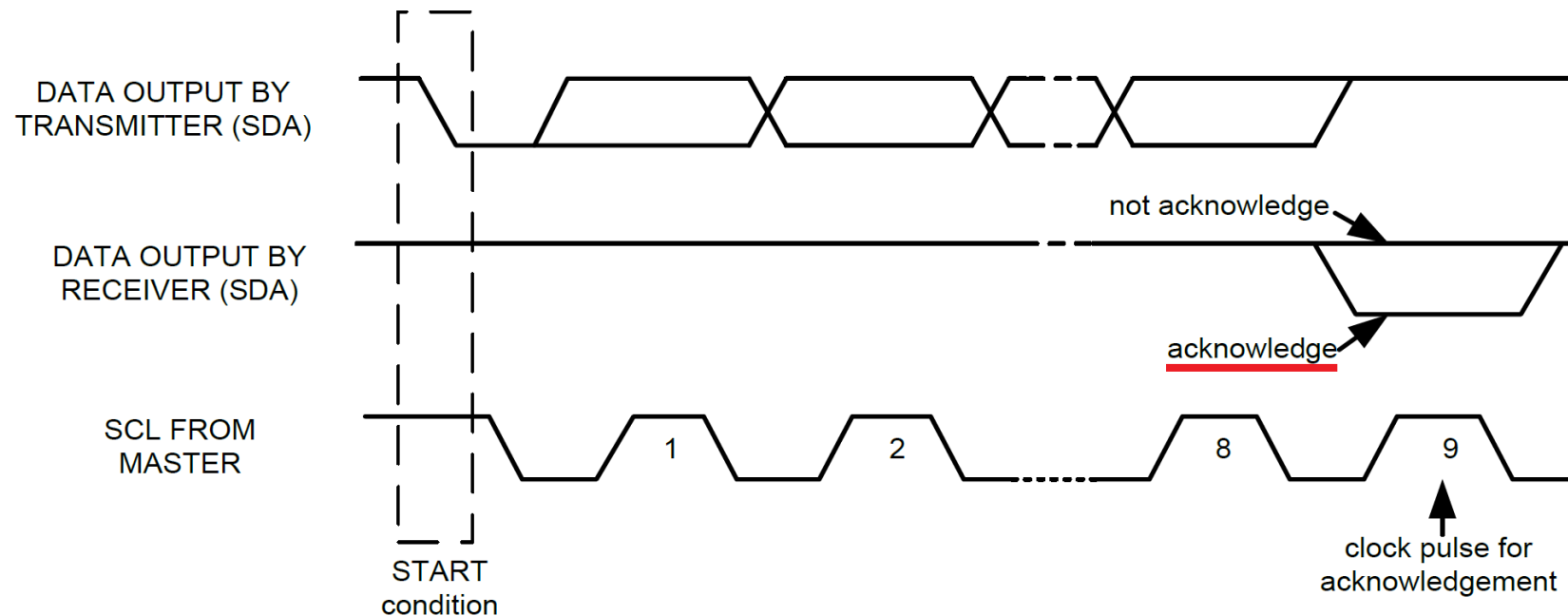
- De open drain (of open collector) outputs moeten aangewend worden om de I²C-communicatie te realiseren.
- Daarbij zijn de **start- en stopconditie** de basis. Daartussen heb je natuurlijk ook nog de verzonden **data** en de **acknowledge** (of not acknowledge).



MPU-9250

Data Format / Acknowledge

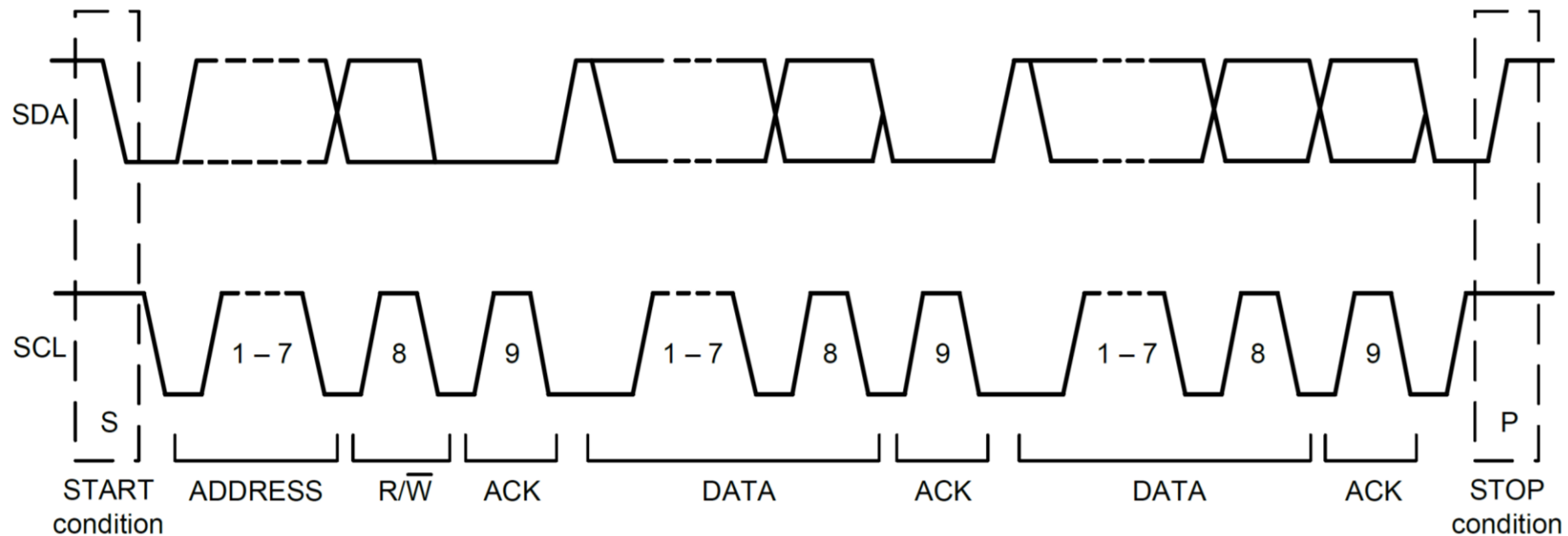
I²C data bytes are defined to be 8-bits long. There is no restriction to the number of bytes transmitted per data transfer. Each byte transferred must be followed by an acknowledge (ACK) signal. The clock for the acknowledge signal is generated by the master, while the receiver generates the actual acknowledge signal by pulling down SDA and holding it low during the HIGH portion of the acknowledge clock pulse.



Acknowledge on the I²C Bus

MPU-9250

Een **volledig** stuk I²C-communicatie kan er dan zo uitzien...



Complete I²C Data Transfer

Je kan data **schrijven** van microcontroller (master) naar sensor (slave).

Single-Byte Write Sequence

S = start, AD = slave address, W = write bit (0), R = read bit (1), ACK = acknowledge, RA = (internal) register address, P = stop, NACK = Not ACK, ...

Master	S	AD+W		RA		DATA		P
Slave			ACK		ACK		ACK	

Burst Write Sequence

Master	S	AD+W		RA		DATA		DATA		P
Slave			ACK		ACK		ACK		ACK	

MPU-9250

Maar je kan ook data **lezen** van sensor (slave) naar microcontroller (master). Merk op dat je **toch eerst een 'schrijfgedeelte'** hebt!

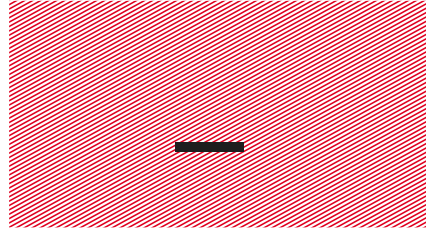
Single-Byte Read Sequence

S = start, AD = slave address, W = write bit (0), R = read bit (1), ACK = acknowledge, RA = (internal) register address, P = stop, NACK = Not ACK, ...

Master	S	AD+W		RA		S	AD+R			NACK	P
Slave			ACK		ACK			ACK	DATA		

Burst Read Sequence

Master	S	AD+W		RA		S	AD+R			ACK		NACK	P
Slave			ACK		ACK			ACK	DATA		DATA		



MPU9250

inlezen via polling



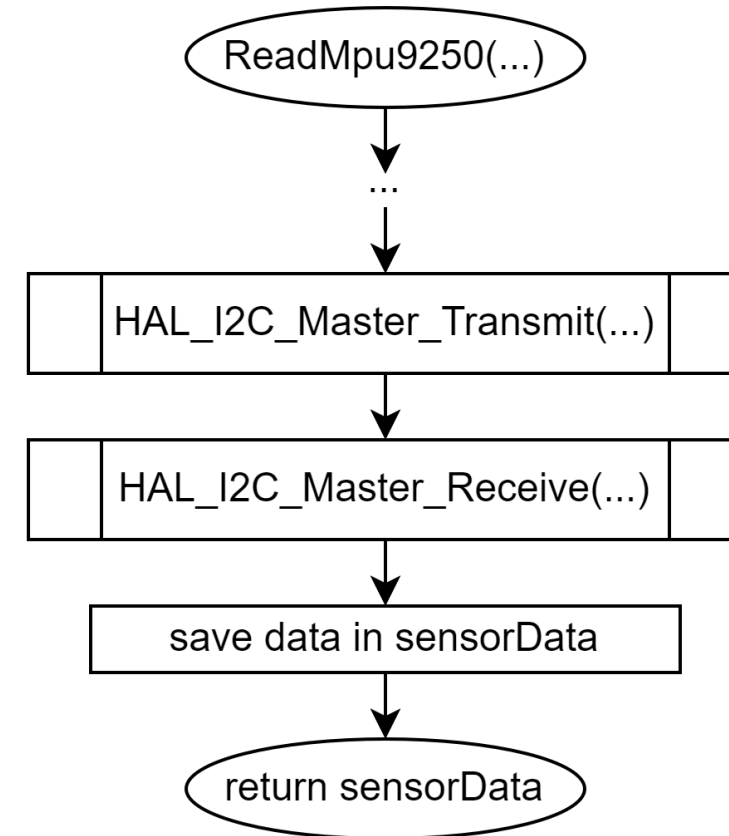
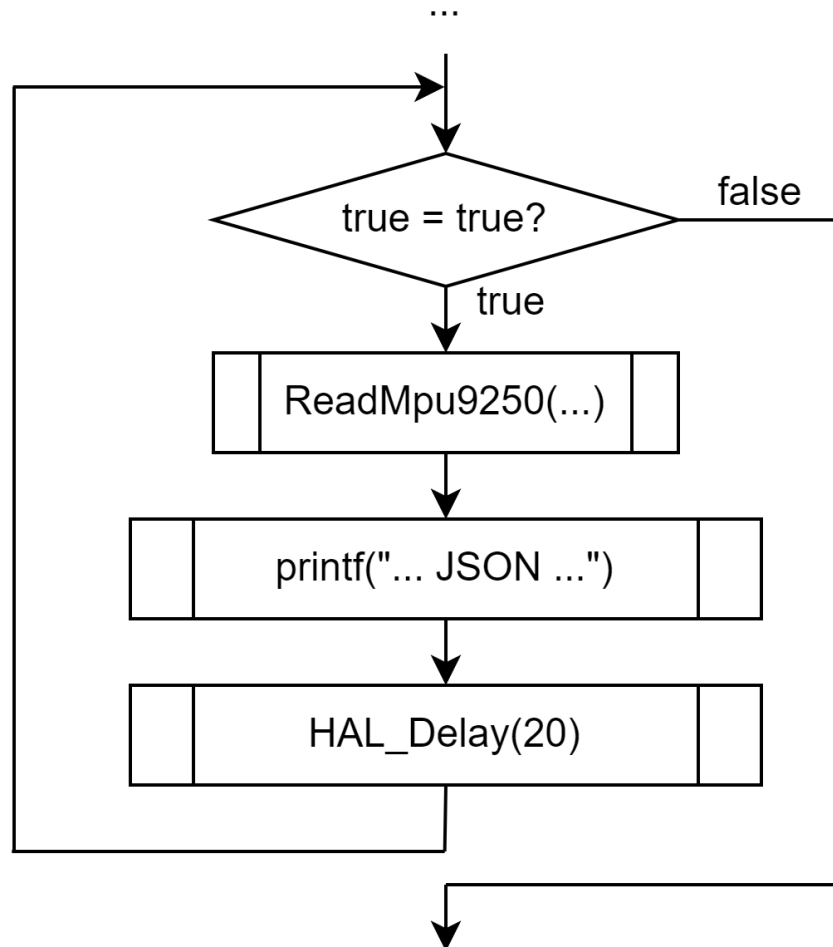
Eerder werd reeds aangehaald dat we op drie manieren kunnen communiceren over digitale bussen:

1. via **polling** (= telkens vragen of we een stap verder kunnen).
2. via interrupts (= we krijgen een melding als we de volgende stap moeten zetten).
3. via DMA (= we krijgen een melding als de laatste stap net gezet is).

Polling is de meest eenvoudige, maar minste efficiënte.

MPU-9250 inlezen via polling

OPM: hier zijn de transmit en receive functies 'blokkerend'!



MPU-9250 inlezen via polling

```
115 while (1)
116 {
117     // IMU uitlezen en JSON dumpen.
118     sensordata = ReadMpu9250(&hi2c1);
119     printf("{\"accx\":%d,\"accy\":%d,\"accz\":%d,\"gyrox\":%d,\"gyroy\":%d,\"gyroz\":%d}\\r\\n",
120         sensordata.accX, sensordata.accY, sensordata.accZ, sensordata.gyroX,
121         sensordata.gyroY, sensordata.gyroZ);
122
123     HAL_Delay(20);
```


MPU-9250 inlezen via polling

```
59 Mpu9250 ReadMpu9250(I2C_HandleTypeDef* hi2c)
60 {
61     Mpu9250 sensordata;
62     uint8_t data[14];
63
64     // Register 0x3B: Acc X High byte.
65     data[0] = 0x3B;
66     HAL_I2C_Master_Transmit(hi2c, (SLAVE_ADDRESS_GYRO_ACC << 1), data, 1, 100);
67
68     // 14 bytes aan IMU-data lezen.
69     HAL_I2C_Master_Receive(hi2c, (SLAVE_ADDRESS_GYRO_ACC << 1), data, 14, 100);
70
71     // Waardes uitrekenen.
72     sensordata.accX = (int16_t)(data[0] * 256 + data[1]);
73     sensordata.accY = (int16_t)(data[2] * 256 + data[3]);
74     sensordata.accZ = (int16_t)(data[4] * 256 + data[5]);
75     sensordata.temp = (int16_t)(data[6] * 256 + data[7]);
76     sensordata.gyroX = (int16_t)(data[8] * 256 + data[9]);
77     sensordata.gyroY = (int16_t)(data[10] * 256 + data[11]);
78     sensordata.gyroZ = (int16_t)(data[12] * 256 + data[13]);
79
80     return sensordata;
81 }
```

MPU-9250 inlezen via polling

Hoe kan je weten dat je met 'blokkerende' functies werkt? **Zoek het op** in de HAL User Manual (UM1884).

HAL_I2C_Master_Transmit

Function name

HAL_StatusTypeDef HAL_I2C_Master_Transmit (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t Timeout)

Function description

Transmits in master mode an amount of data in blocking mode.

Parameters

- **hi2c**: Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress**: Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData**: Pointer to data buffer
- **Size**: Amount of data to be sent
- **Timeout**: Timeout duration

Return values

- **HAL**: status

MPU-9250 inlezen via polling

HAL_I2C_Master_Receive

Function name

`HAL_StatusTypeDef HAL_I2C_Master_Receive (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size, uint32_t Timeout)`

Function description

Receives in master mode an amount of data in blocking mode.

Parameters

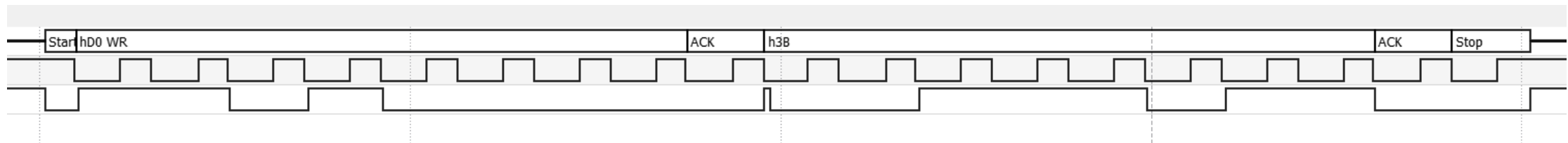
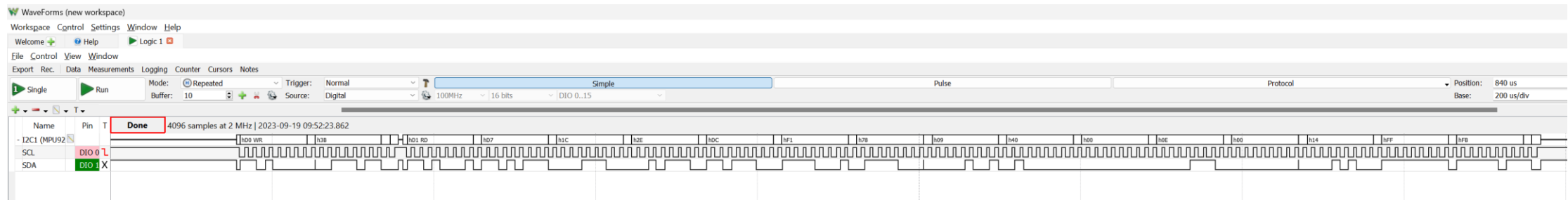
- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent
- **Timeout:** Timeout duration

Return values

- **HAL:** status

MPU-9250 inlezen via polling

Kan je de voorspelde communicatie, effectief ontdekken via een USB-oscilloscoop (Analog Discovery 2)? Of op de onderstaande screenshots? **Doen!**

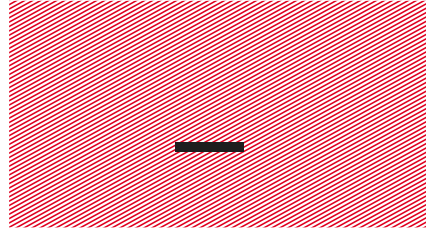


Het 7-bit MPU-adres is 0x68. Als je dat één bit naar links opschuift kom je uit op 0xD0... Daarbij komt dan de lees- of schrijfbits.

MPU-9250 inlezen via polling

De firmware stuurt de gemeten data in JSON-formaat door via USART2. Met die info kan je dus bijvoorbeeld een C#-applicatie maken om eender wat te doen...

```
COM3 - PuTTY
{"accx":-9940,"accy":12604,"accz":-3416,"gyrox":28,"gyroy":23,"gyroz":-14}
{"accx":-9896,"accy":12620,"accz":-3412,"gyrox":18,"gyroy":0,"gyroz":-18}
{"accx":-9864,"accy":12532,"accz":-3480,"gyrox":10,"gyroy":21,"gyroz":-3}
{"accx":-9936,"accy":12604,"accz":-3416,"gyrox":1,"gyroy":16,"gyroz":-3}
{"accx":-9944,"accy":12592,"accz":-3432,"gyrox":-3,"gyroy":12,"gyroz":-2}
{"accx":-10004,"accy":12660,"accz":-3472,"gyrox":-2,"gyroy":7,"gyroz":-5}
{"accx":-9920,"accy":12644,"accz":-3508,"gyrox":1,"gyroy":19,"gyroz":-7}
{"accx":-9992,"accy":12596,"accz":-3476,"gyrox":3,"gyroy":11,"gyroz":-19}
{"accx":-9944,"accy":12560,"accz":-3460,"gyrox":12,"gyroy":18,"gyroz":-14}
{"accx":-9932,"accy":12628,"accz":-3564,"gyrox":23,"gyroy":11,"gyroz":-14}
{"accx":-9928,"accy":12532,"accz":-3540,"gyrox":24,"gyroy":14,"gyroz":-5}
```



MPU9250 inlezen via DMA



MPU-9250 inlezen via DMA

STM32CubeMX CubeMX - Template - Nucleo-L476RG.ioc*: STM32L476RGTx NUCLEO-L476RG

File Window Help myST

Home STM32L476RGTx - NUCLEO-L476RG CubeMX - Template - Nucleo-L476RG.ioc - Pinout & Configuration GENERATE CODE

Pinout & Configuration Clock Configuration Project Manager Tools

Software Packs Pinout

I2C1 Mode and Configuration

Mode

I2C I2C

Configuration

Reset Configuration

NVIC Settings DMA Settings GPIO Settings

Parameter Settings User Constants

DMA Request	Channel	Direction	Priority
I2C1_RX	DMA1 Channel 7	Peripheral To Memory	Medium

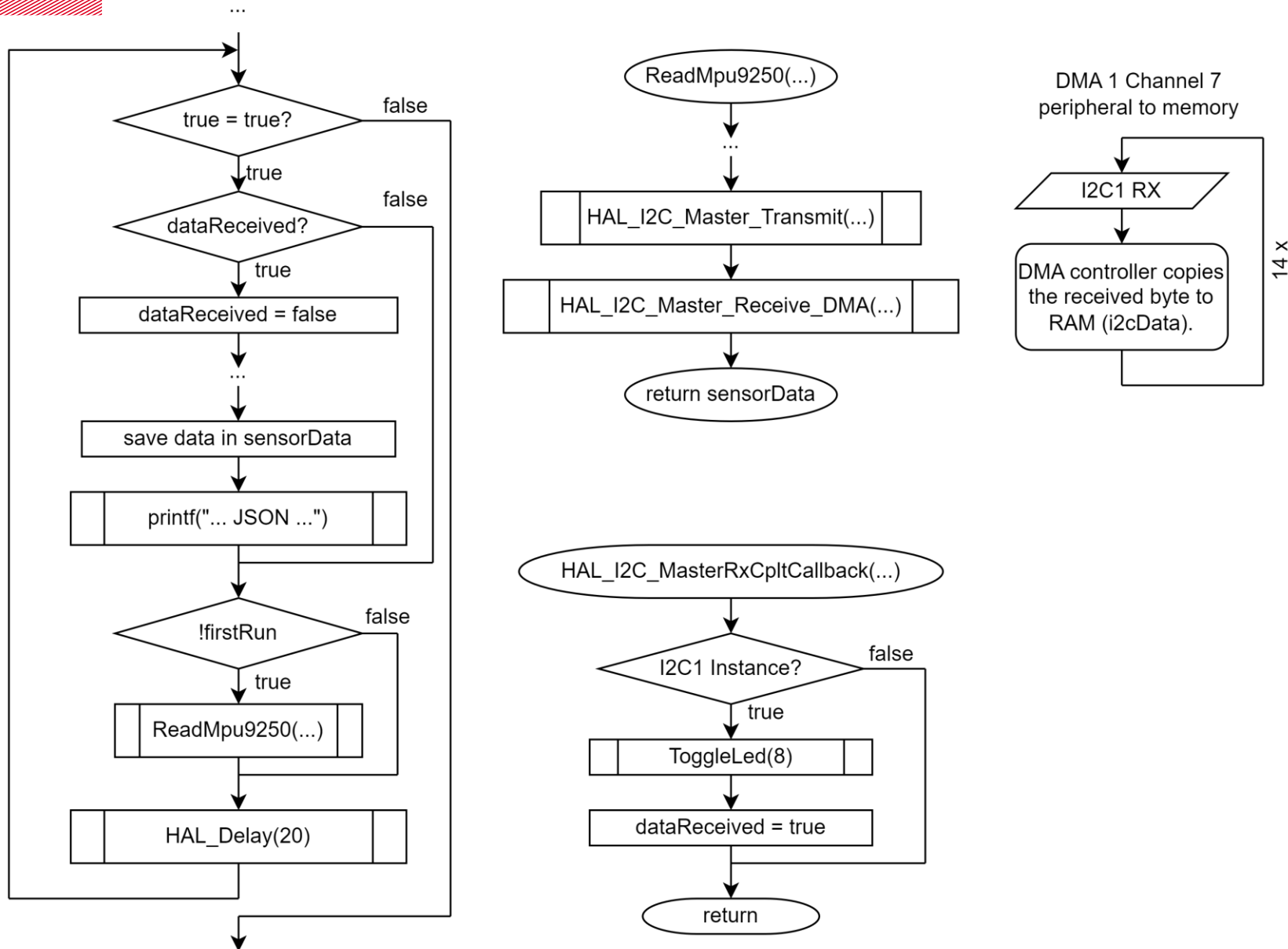
Add Delete

Pinout view System view

STM32L476RGTx LQFP64

The screenshot shows the STM32CubeMX interface. On the left, the 'Categories' list is expanded to 'Connectivity', and 'I2C1' is selected. The main panel shows the 'I2C1 Mode and Configuration' window. Under the 'Configuration' tab, 'DMA Settings' is checked. A table lists the DMA configuration: I2C1_RX is connected to DMA1 Channel 7, with a direction of 'Peripheral To Memory' and a priority of 'Medium'. A red arrow points to the 'DMA1 Channel 7' entry. On the right, the 'Pinout view' shows the STM32L476RGTx LQFP64 package with various pins labeled, including I2C1 pins (SDA1, SCL1) and DMA pins (PB7, PB8, PB9, PB10, PB11, PB12, PB13, PB14, PB15).

MPU-9250 inlezen via DMA



MPU-9250 inlezen via DMA

```
127 while (1)
128 {
129     // Is er data ontvangen via DMA, toon ze...
130     if(dataReceived)
131     {
132         dataReceived = false;
133
134         if(firstRun)
135         {
136             firstRun = false;
137
138             // Ontvangen Who-am-I-data uitlezen.
139             whoAmI = i2cData[0];
140
141             printf("Who am I: %d.\r\n", whoAmI);
142         }
143         else
144         {
145             // Ontvangen IMU-data uitlezen.
146             sensordata.accX = (int16_t) (i2cData[0] * 256 + i2cData[1]); // Casten naar 16 bit signed integer.
147             sensordata.accY = (int16_t) (i2cData[2] * 256 + i2cData[3]);
148             sensordata.accZ = (int16_t) (i2cData[4] * 256 + i2cData[5]);
149             sensordata.temp = (int16_t) (i2cData[6] * 256 + i2cData[7]);
150             sensordata.gyroX = (int16_t) (i2cData[8] * 256 + i2cData[9]);
151             sensordata.gyroY = (int16_t) (i2cData[10] * 256 + i2cData[11]);
152             sensordata.gyroZ = (int16_t) (i2cData[12] * 256 + i2cData[13]);
153
154             printf("{ \"accx\":%d, \"accy\":%d, \"accz\":%d, \"gyrox\":%d, \"gyroy\":%d, \"gyroz\":%d }\r\n",
155                 sensordata.accX, sensordata.accY, sensordata.accZ, sensordata.gyroX, sensordata.gyroY, sensordata.gyroZ);
156         }
157     }
158
159     // IMU uitlezen starten via DMA en JSON dumpen indien de 'Who am I' reeds gekend is.
160     if(!firstRun)
161         ReadMpu9250(&hi2c1);
162
163     // Uitlezen van de I²C-bus duurt ongeveer 1,5ms (2 + 14 bytes).
164     HAL_Delay(20);
165 }
```

MPU-9250 inlezen via DMA

```
58 void ReadMpu9250(I2C_HandleTypeDef* hi2c)
59 {
60     extern uint8_t i2cData[NUMBER_OF_BYTES_TO_RECEIVE];
61
62     // Register 0x3B: Acc X High byte.
63     i2cData[0] = 0x3B;
64     HAL_I2C_Master_Transmit(hi2c, (SLAVE_ADDRESS_GYRO_ACC << 1), i2cData, 1, 100);
65
66     // 14 bytes aan IMU-data lezen.
67     HAL_I2C_Master_Receive_DMA(hi2c, (SLAVE_ADDRESS_GYRO_ACC << 1), i2cData, 14);
68 }
```

```
461 // MERK OP: om met HAL I2C en DMA te kunnen werken, moet in STM32CubeMX 'I2C1 event interrupt' aangevinkt staan!
462 // Anders wordt HAL_I2C_MasterRxCallback() niet opgeroepen.
463 void HAL_I2C_MasterRxCallback(I2C_HandleTypeDef *hi2c)
464 {
465     // Is de ontvangst van I2C1?
466     if(hi2c->Instance == I2C1)
467     {
468         ToggleLed(8); // Teken van leven geven...
469         dataReceived = true; // Melden aan de hoofdloop.
470     }
471 }
```

Houd het hier **kort!**

MPU-9250 inlezen via DMA

Hoe kan je weten dat je met 'niet-blokkerende' functies werkt?
Zoek het op in de HAL User Manual (UM1884).

HAL_I2C_Master_Receive_DMA

Function name

HAL_StatusTypeDef HAL_I2C_Master_Receive_DMA (I2C_HandleTypeDef * hi2c, uint16_t DevAddress, uint8_t * pData, uint16_t Size)

Function description

Receive in master mode an amount of data in non-blocking mode with DMA.

Parameters

- **hi2c:** Pointer to a I2C_HandleTypeDef structure that contains the configuration information for the specified I2C.
- **DevAddress:** Target device address: The device 7 bits address value in datasheet must be shifted to the left before calling the interface
- **pData:** Pointer to data buffer
- **Size:** Amount of data to be sent

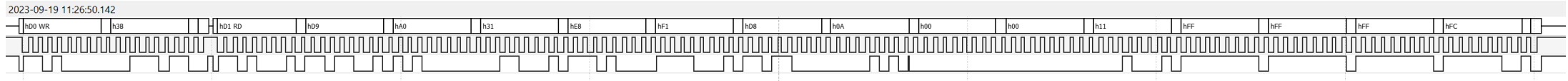
Return values

- **HAL:** status

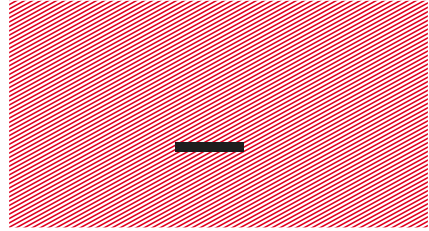


MPU-9250 inlezen via DMA

De scoopbeelden moet er gelijkaardig uitzien als die van de 'polling' versie, **maar de CPU heeft zelf veel meer tijd** om nuttige berekeningen te maken!



De code in de main() ziet er wel complexer uit, maar toch benut deze DMA-versie de microcontroller op een veel betere manier.



'Head tracking' demo





Head tracking demo



<https://www.youtube.com/watch?v=G5nRet-r62s>



Head tracking demo

In sommige gevallen kan het handig (of leuk) zijn dat een **camera mee beweegt met de draaiing van je hoofd**. Zeker in FPV-situaties kan dat van belang zijn (FPV-vliegen, -rijden, -varen,...).

Kan je met behulp van een IMU, een modelbouwservo en onze Nucleo, zo'n wannabe 'head tracking' systeem maken? Natuurlijk ;-)

Wat hebben we nodig? **I²C**-bus (best met DMA), Timer met **PWM**-uitgang, camera, microcontroller, ...

Head tracking demo

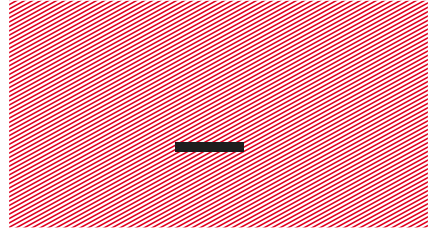
```
569 // Verwerken van de sensor data nadat de DMA een signaal (interrupt) geeft.
570 // MERK OP: om met HAL I²C en DMA te kunnen werken, moet in STM32CubeMX 'I2C1 event interrupt' aangevinkt staan!
571 // Anders wordt HAL_I2C_MasterRxCpltCallback() niet opgeroepen.
572 // Het starten van de meting gebeurt in de SysTick_Handler() iedere 4ms.
573 void HAL_I2C_MasterRxCpltCallback(I2C_HandleTypeDef *hi2c)
574 {
575     // Is de ontvangst van I2C1?
576     if(hi2c->Instance == I2C1)
577     {
578         // Teken van leven geven...
579         ToggleLed(8);
580
581         // Gyro Y-data verwerken.
582         sensordata.gyroY = (int16_t)(i2cData[10] * 256 + i2cData[11]);
583
584         // Nieuwe hoek berekenen: hoek = draaisnelheid * draaitijd.
585         // TODO: nog beter zou zijn dat er met SENSOR FUSION gewerkt wordt (gyroscop + magnetometer).
586         // Op die manier kan je de 'drift' uit het signaal filteren.
587         angle = angle - ((float)(sensordata.gyroY - gyroYOffset)/16.375f) * (float)SENSOR_READ_INTERVAL/1000.0f;
588         // of
589         //angle = angle - ((float)(sensordata.gyroY - gyroYOffset)/5458.3f);
590
591         // PWM-waarde voor de servo berekenen, herschalen en begrenzen.
592         pwmValue = PWM_MINIMUM + PWM_MIDPOINT + angle * ANGLE_TO_PWM_SCALE - servoTrim;
593         if(pwmValue < PWM_MINIMUM)
594             pwmValue = PWM_MINIMUM;
595         if(pwmValue > PWM_MAXIMUM)
596             pwmValue = PWM_MAXIMUM;
597
598         // PWM-waarde opslaan in het CCR2-register (als het kalibreren voorbij is).
599         if(calibrationOk)
600             __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, pwmValue);
601     }
602 }
```

4ms (zie stm32l4xx_it.c)

—

Head tracking demo





Theorieopdracht



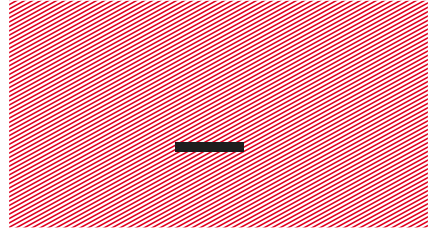


Theorieopdracht

Maak gebruik van de demo code i.v.m. de MPU-9250, de modelbouwservo en DMA (head tracking) om enkele vragen te beantwoorden.

De code heeft als naam: 'Nucleo-L476RG - DMA - I2C to memory - MPU9250 and servo (head tracking)'.

Zie: "**Theorieopdracht 4 – I2C, MPU-9250, ...**".



Uitbreidingsopdracht

(vrijblijvend)





Uitbreidingsopdracht (vrijblijvend)

Klaar?

Vraag een Analog Discovery 2 USB oscilloscoop aan de lector en visualiseer eender welke I²C-communicatie. Je zal daarvoor dus ook een I²C-sensor/device en 'jumper wires' nodig hebben.

Let goed op tijdens het verbinden van de GND en 3.3V! Foute aansluitingen leiden tot defecte sensoren!